# GARAGE PREMIUM MODERNO

## Informe técnico del servicio REST y persistencia

Hito 2 · Proyecto semestral 2026
INGT1003 · Universidad Mayor

Joel Trujillo · JoelTrujillo2003
Joaquín Guajardo · JoaquinG

Versión: 20 de octubre de 2026. El documento describe el código y la base comprobados al cierre de esta versión. La demostración HTTP con una cuenta de prueba debe ejecutarse en sala.

# 1. Resumen y alcance

El Hito 1 presentó una web para registrar autos y motos en Chile y obtener orientación inicial de mantención. Este hito construye y documenta el servicio del lado del servidor: una API REST para vehículos y registros de mantención, con las cuatro operaciones de cada recurso y persistencia PostgreSQL en Supabase. El sitio ya publicado continúa siendo la interfaz principal; la colección Postman permite evaluar el servicio directamente.

## Objetivo verificable

Una persona autenticada puede crear, listar, leer, actualizar y eliminar sus vehículos y registros de mantención. Cada operación responde con un código HTTP congruente y conserva los datos entre reinicios del servidor. El acceso entre cuentas se limita tanto en las consultas como con políticas RLS de la base.

| Pieza | Ubicación y función |
|---|---|
| Servidor | server.js inicia Express y monta los routers del dominio. |
| Controladores | src/controllers: traducen HTTP a llamadas del servicio. |
| Servicios | src/services: validan y aplican reglas del catálogo, plan y propiedad. |
| Repositorios | src/repositories: consultan PostgREST de Supabase con el JWT del usuario. |
| Base | supabase/migrations y db: PostgreSQL, claves, restricciones y políticas RLS. |

## Estado de verificación

Se ejecutaron 38 pruebas automatizadas, incluida una prueba HTTP integral de los dos CRUD con Supabase simulado. La migración 004 se comprobó primero en PostgreSQL embebido y luego se aplicó en el proyecto Supabase: la tabla maintenance_records y sus ocho columnas fueron verificadas en el editor SQL. Estas pruebas no reemplazan las cuatro peticiones reales de la exposición.

# 2. Trazabilidad con el Hito 1

El Hito 1 definió un recorrido: crear cuenta, escoger tipo/marca/modelo/año, guardar kilometraje y consultar una guía inicial. Sus datos de vehículo ya persistían, pero la API propia solo permitía crear y listar; tampoco existía un historial editable de mantenciones. El Hito 2 convierte esa ficha en un recurso REST completo y agrega el historial relacionado.

| Hito 1 | Cambio del Hito 2 | Impacto |
|---|---|---|
| Ficha del vehículo y Mi garage | GET por id, PUT y DELETE además de GET lista y POST. | La ficha puede corregirse y retirarse mediante el servidor. |
| Guía inicial de mantención | Nuevo recurso maintenance-records con CRUD. | La persona puede registrar revisiones, servicios y reparaciones con fecha y km. |
| Supabase Auth y PostgreSQL | Clave compuesta, cascada y RLS para el historial. | Un registro no puede apuntar a vehículo ajeno; borrar la ficha borra su historial. |
| Servidor Express con rutas principales | Controlador, servicio y repositorio separados para ambos recursos. | Las reglas no quedan escondidas en el archivo de rutas. |
| Prototipo web publicado | Contrato de API, migración y colección de pruebas. | El servidor se puede demostrar independientemente de la interfaz. |

## Decisión de alcance

Se eligieron vehículos y mantenciones porque forman el núcleo del problema del proyecto. El módulo de IA y la propuesta Premium siguen disponibles, pero no se presentan como sustitutos de la persistencia y del CRUD exigidos. La integración de cobros no forma parte de la demostración del Hito 2.

# 3. Contrato REST: vehículos

Base local: http://localhost:3000. Todas las rutas requieren Authorization: Bearer <access_token> de Supabase Auth. POST y PUT usan JSON. El servidor determina user_id desde el token; no acepta elegir el dueño desde el cuerpo.

| Método | Ruta | Entrada | Éxito | Errores |
|---|---|---|---|---|
| GET | /api/vehicles | Sin cuerpo | 200 lista | 401 / 503 |
| GET | /api/vehicles/:id | UUID | 200 objeto | 400 / 401 / 404 |
| POST | /api/vehicles | Ficha completa | 201 objeto + Location | 400 / 401 / 403 |
| PUT | /api/vehicles/:id | UUID + ficha completa | 200 objeto | 400 / 401 / 403 / 404 |
| DELETE | /api/vehicles/:id | UUID | 204 sin cuerpo | 400 / 401 / 404 |

## Cuerpo de ejemplo

{"type":"Moto","brand":"Bajaj","model":"Pulsar RS200","year":2020,"mileage":10000}

El catálogo valida la relación tipo-marca-modelo-año. El kilometraje es entero de 0 a 999.999 y admite la cadena chilena "10.000". El plan Free conserva el límite de dos vehículos y los modelos marcados Premium requieren suscripción vigente. PUT reemplaza los cinco campos editables; DELETE elimina también las mantenciones asociadas por cascada.

## Ejemplo de respuesta 201

{"vehicle":{"id":"<uuid>","type":"Moto","brand":"Bajaj",...}}
Location: /api/vehicles/<uuid>

# 4. Contrato REST: mantenciones

| Método | Ruta | Entrada | Éxito | Errores |
|---|---|---|---|---|
| GET | /api/maintenance-records | ?vehicle_id=UUID opcional | 200 lista | 400 / 401 / 503 |
| GET | /api/maintenance-records/:id | Id entero | 200 objeto | 400 / 401 / 404 |
| POST | /api/maintenance-records | Registro completo | 201 objeto + Location | 400 / 401 / 404 |
| PUT | /api/maintenance-records/:id | Id + registro completo | 200 objeto | 400 / 401 / 404 |
| DELETE | /api/maintenance-records/:id | Id entero | 204 sin cuerpo | 400 / 401 / 404 |

## Cuerpo de ejemplo

{"vehicle_id":"<uuid del vehículo>","category":"service",
"performed_on":"2026-08-10","mileage":10000,
"notes":"Cambio de aceite y revisión general."}

Las categorías permitidas son inspection, service y repair. La fecha debe existir y no estar en el futuro según America/Santiago. El kilometraje admite 0 a 999.999; notes requiere 5 a 1.000 caracteres. Antes de escribir se comprueba que el vehículo pertenezca a la cuenta; la clave foránea compuesta repite esta condición en PostgreSQL.

## Contrato de errores

Formato común: {"error":"Mensaje legible","code":"codigo_estable"}. 400: parámetros o cuerpo inválidos. 401: sesión ausente o vencida. 403: regla de plan. 404: recurso inexistente o ajeno, sin revelar datos de terceros. 503: servicio o migración indisponible. Los fallos no se anuncian con 200. El contrato completo con las rutas complementarias está en docs/hito2/API.md.

# 5. Capas y flujo de una petición

| Etapa | Responsabilidad concreta | Ejemplo POST /api/maintenance-records |
|---|---|---|
| Router / controlador | Autenticar, leer parámetros, elegir código y cabeceras HTTP. | Devuelve 201 y Location; no consulta directamente tablas. |
| Servicio | Validar fecha, categoría, kilometraje y dueño; decidir error de negocio. | Rechaza fecha futura con 400 y vehículo ajeno con 404. |
| Repositorio | Construir petición PostgREST y filtrar por user_id. | Inserta en maintenance_records y pide la fila creada. |
| PostgreSQL | Persistir y defender integridad, claves y acceso. | FK (vehicle_id,user_id), CHECK y RLS por auth.uid(). |

## Secuencia

Cliente → Express → authenticateRequest → controlador → servicio → repositorio → PostgREST → PostgreSQL. La respuesta vuelve por el mismo camino; el middleware de errores decide el estado final. El repositorio utiliza el JWT del usuario para que la política de filas se aplique de manera efectiva, y además filtra por user_id en la consulta.

## Dónde vive cada regla

La validación del catálogo y del plan está en el servicio de vehículos; la validación de mantenciones, en su servicio. Los controladores solo traducen HTTP. Las consultas de lectura y escritura están encapsuladas en repositorios. Las invariantes que deben sobrevivir a cualquier cliente -propietario, rango de km, categoría y relación vehículo-mantención- también se expresan en SQL.

# 6. Persistencia y modelo de datos

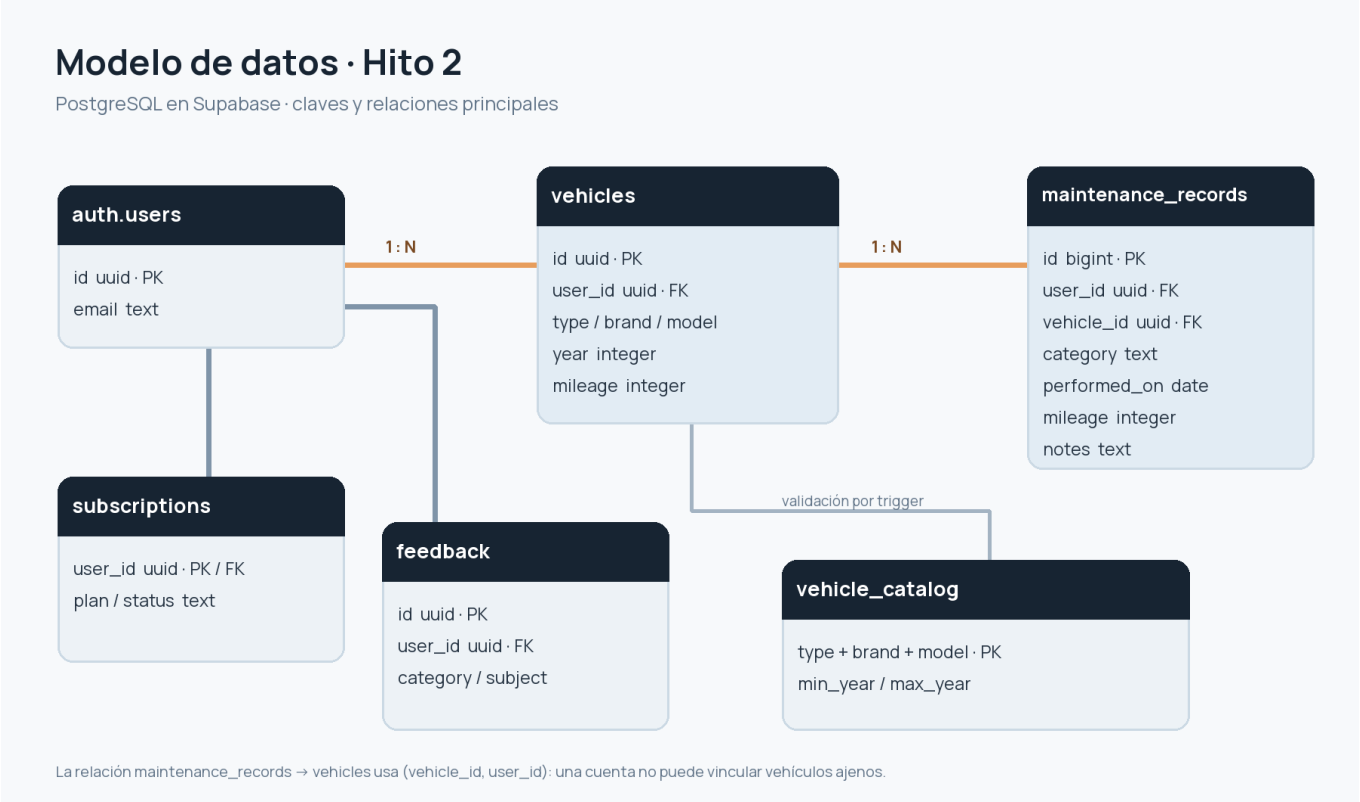



Figura 1. Modelo de datos del proyecto. El nuevo historial se une a vehículos por id y user_id. Fuente: elaboración propia a partir de las migraciones SQL.

## Motor y creación

Supabase utiliza PostgreSQL. El script db/crear_desde_cero.sql concatena las migraciones 001 a 004 para un proyecto nuevo. Para el proyecto existente se ejecutó solamente la 004. En la base real vehicles.id es UUID; la migración usa vehicle_id UUID, diferencia comprobada antes de aplicar el cambio. La tabla maintenance_records ya se verificó en la base remota.

La clave foránea compuesta (vehicle_id, user_id) referencia vehicles(id, user_id) y usa ON DELETE CASCADE. Esto impide asociar el historial de una cuenta a un vehículo de otra y evita registros huérfanos al borrar la ficha. Se añadieron índices para consultar por dueño/fecha y vehículo/fecha.

# 7. Seguridad y manejo de credenciales

Las rutas exigen el access token de Supabase. authenticateRequest obtiene el usuario validado; el servidor ignora un user_id escrito por el cliente. Las políticas RLS de vehicles y maintenance_records solo permiten seleccionar, insertar, actualizar y borrar filas cuyo user_id coincide con auth.uid(). La tabla nueva no da permisos de lectura o escritura al rol anon.

| Dato | Dónde se configura | Regla |
|---|---|---|
| URL y clave pública Supabase | Variable de entorno y configuración pública necesaria para Auth | La clave pública no concede acceso a otras filas sin JWT y políticas. |
| Clave secreta Supabase | Variable de entorno solo en servidor | Nunca dentro de public/ ni en el ZIP. |
| Gemini API key | Variable de entorno solo en servidor | El navegador llama al servidor; no recibe esta clave. |
| Contraseñas y tokens de demostración | Variables locales de Postman | No se exportan ni se suben al repositorio. |

## Configuración repetible

La raíz contiene .env.example con nombres y valores de ejemplo. .gitignore excluye .env, credenciales locales y node_modules. Si una credencial real se hubiese subido alguna vez a Git, borrarla del último commit sería insuficiente: corresponde rotarla y revisar el historial. La colección exportada deja vacíos email, password y access_token.

# 8. Tecnologías y arquitectura

## Node.js y Express

El producto ya usaba JavaScript en navegador y Node.js en servidor. Mantener el lenguaje evita duplicar reglas de validación y reduce el trabajo de instalación para un equipo de dos integrantes. Express ofrece métodos y routers asociados a los verbos HTTP sin imponer un ORM; el costo es definir explícitamente validación, errores y estructura de capas. Frente a una reescritura con otro framework, el equipo priorizó hacer verificable el contrato sobre una migración tecnológica que no agregaría valor al hito.

## Supabase, PostgreSQL y PostgREST

PostgreSQL aporta claves foráneas, restricciones CHECK, índices y transacciones. Supabase añade autenticación y políticas RLS ya usadas por el prototipo. PostgREST permite consultas y mutaciones desde los repositorios conservando el JWT del usuario; Prefer: return=representation confirma la fila modificada. El costo es la dependencia operativa de Supabase y la necesidad de mantener la migración SQL junto con el código.

## Monolito modular frente a microservicios

Se mantiene un único servidor Express desplegado en Render, con módulos internos separados. Para dos personas y una carga de presentación pequeña, un despliegue y un contrato interno claro cuestan menos que coordinar varios servicios, redes, autenticación duplicada y observabilidad distribuida. El costo aceptado es que un fallo del proceso afecta a toda la API y que escalar IA y CRUD por separado requeriría trabajo adicional. Revisaríamos la decisión si el tráfico, los tiempos de Gemini o equipos independientes exigieran ciclos de despliegue y escalado distintos; antes mediríamos esos límites.

# 9. Pruebas y demostración en vivo

La colección docs/hito2/Garage-Hito2.postman_collection.json contiene autenticación y peticiones guardadas en orden. El bloque principal ejecuta POST, GET, PUT y DELETE de vehículos con un UUID capturado automáticamente; un bloque adicional hace lo mismo con mantenciones. Hay ejemplos de 400, 401 y 404. Para presentar se requiere una cuenta de prueba y un token vigente, ingresados localmente sin mostrarlos en pantalla.

| Minuto | Acción | Evidencia visible |
|---|---|---|
| 0:00–0:30 | Abrir API y explicar recurso. | Health 200; variables privadas ocultas. |
| 0:30–2:35 | Ejecutar POST, GET, PUT, DELETE de vehículos. | 201 + Location, lectura persistente, 200 modificado, 204 sin cuerpo. |
| 2:35–3:20 | Mostrar mantenciones y un error. | Relación por vehicle_id; 400 o 404 real. |
| 3:20–4:25 | Recorrer controlador, servicio y repositorio. | Una regla real en cada capa. |
| 4:25–5:20 | Mostrar PostgreSQL y migración. | Tabla, FK compuesta, RLS, script. |
| 5:20–6:25 | Justificar stack y monolito modular. | Ventaja, costo y alternativa. |
| 6:25–7:00 | Cambios desde Hito 1 y cierre. | Trazabilidad y límite de alcance. |

La colección puede apuntar al servidor local o público mediante base_url. Antes de exponer en sala, probar todo en otro equipo con npm ci, .env propio y la migración aplicada. No se debe considerar una captura o grabación como sustituto de las peticiones ejecutadas frente al profesor.

# 10. Validación, límites y responsabilidades

| Comprobación | Resultado al cierre |
|---|---|
| Pruebas de Node | 38 de 38 pasaron, incluidas rutas HTTP con Supabase simulado. |
| Migración PostgreSQL embebida | FK compuesta rechaza vehículo ajeno; cascada borra mantenciones. |
| Migración Supabase real | 004 aplicada; ocho columnas de maintenance_records confirmadas en SQL Editor. |
| Ejecución con sesión real | Pendiente de completar en la demostración; la colección no incluye credenciales. |
| Pago Mercado Pago | No está habilitado para la presentación de este hito. |

## Responsabilidades del equipo

Joel Trujillo: servidor, contrato HTTP, validación de respuestas y despliegue. Joaquín Guajardo: modelo de datos, migraciones, colección de pruebas y explicación de persistencia. Ambos deben poder explicar un controlador, un servicio, un repositorio y la decisión de arquitectura si el profesor elige a cualquiera.

## Riesgos que deben comprobar antes de presentar

Un token caducado devuelve 401; una cuenta Free que ya tiene dos vehículos devuelve 403 al crear otro; un modelo Premium sin suscripción devuelve 403. Para demostrar POST se recomienda una cuenta Free nueva y un modelo estándar del catálogo. Render en plan gratuito puede tardar en despertar; abrir /api/health antes del bloque cronometrado. La IA depende de un servicio externo y no es la evidencia del CRUD.

# 11. Referencias técnicas

• Express.js. “Routing”. https://expressjs.com/en/guide/routing/ (consulta: 20-10-2026).

• Supabase. “Row Level Security”. https://supabase.com/docs/guides/database/postgres/row-level-security (consulta: 20-10-2026).

• PostgREST. “Prefer Header”. https://docs.postgrest.org/en/v12/references/api/preferences.html (consulta: 20-10-2026).

• PostgreSQL Global Development Group. “Constraints”. https://www.postgresql.org/docs/current/ddl-constraints.html (consulta: 20-10-2026).

• Proyecto propio. Garage Premium Moderno, código, contrato REST y migraciones del Hito 2. https://github.com/JoelTrujillo2003/ProyectoMantencion (revisión de esta entrega).

## Archivos anexos del repositorio

docs/hito2/API.md contiene la tabla de recursos; db/modelo-datos.png y .mmd, el diagrama; db/crear_desde_cero.sql, la base inicial; supabase/migrations/004_maintenance_records.sql, el cambio incremental; docs/hito2/Garage-Hito2.postman_collection.json, las solicitudes importables; .env.example, la configuración sin secretos.